

Programming in Java

(BCAC 0012)

Module-2

Faculty Name – Dr. Vinod Jain

Syllabus Module 2

Introduction of Classes, Objects and Methods	Defining Class, Adding Variables
	Adding Methods-Default methods, Parameterized methods Creating Objects
	Constructors ,Types Of constructor
	Keyword-this ,super
	Accessing Class Members, final Variables and Methods
	Visibility Control
	Static keyword-variable, class, method,
Inheritance	Inheritance, Member Access, Super Class & subclass
	Types of Inheritance
	Abstract Class, Dynamic method Dispatching
	Method Over Loading & Overriding,
Packages And Interfaces	Defining Packages, Importing Packages, User Defined packages and predefined packages
	Introduction to Interfaces.

Remaining Topics

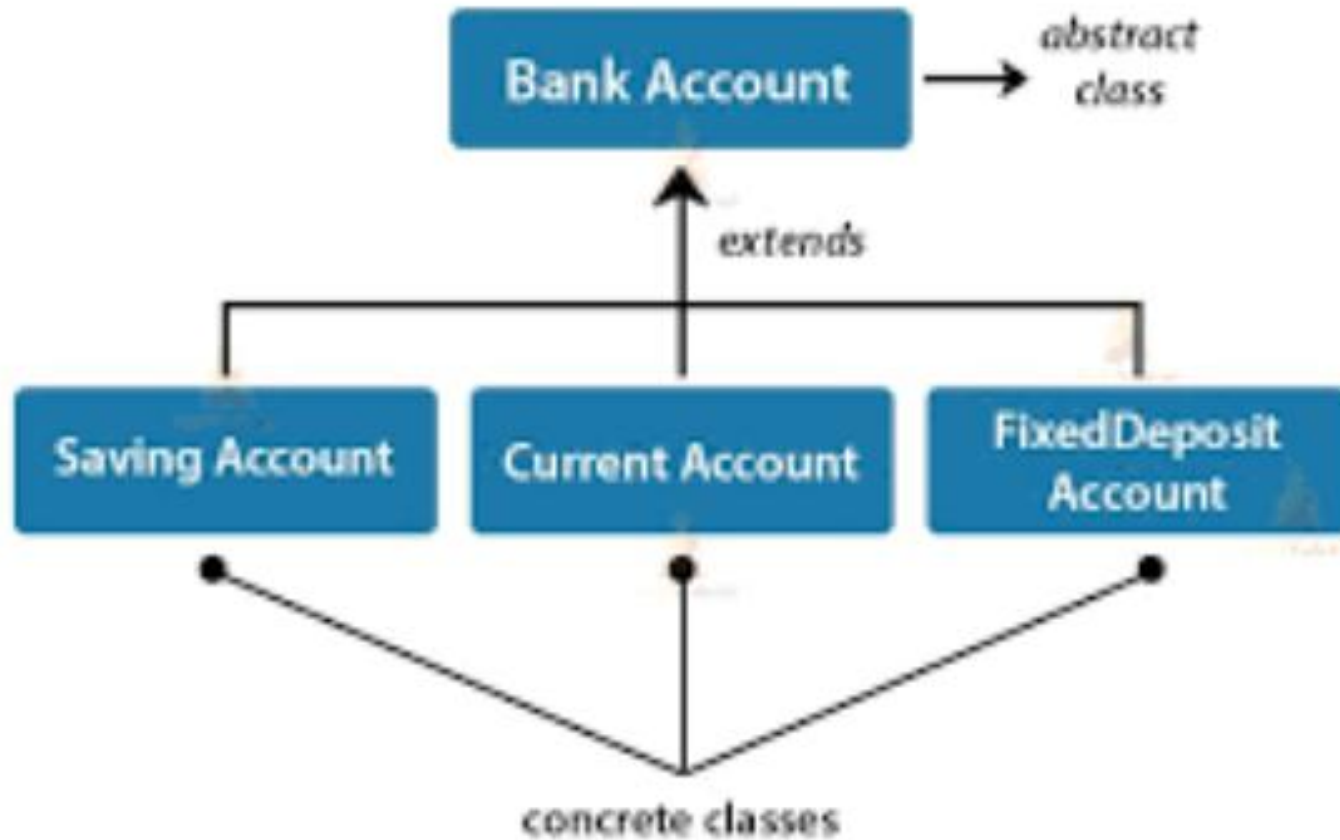
1. Abstract Class
2. Final Variables and Methods

Abstraction

- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.
- Abstraction lets you focus on **what the object does** instead of how it does it.

Abstract Class

Example of Abstract Class in Java



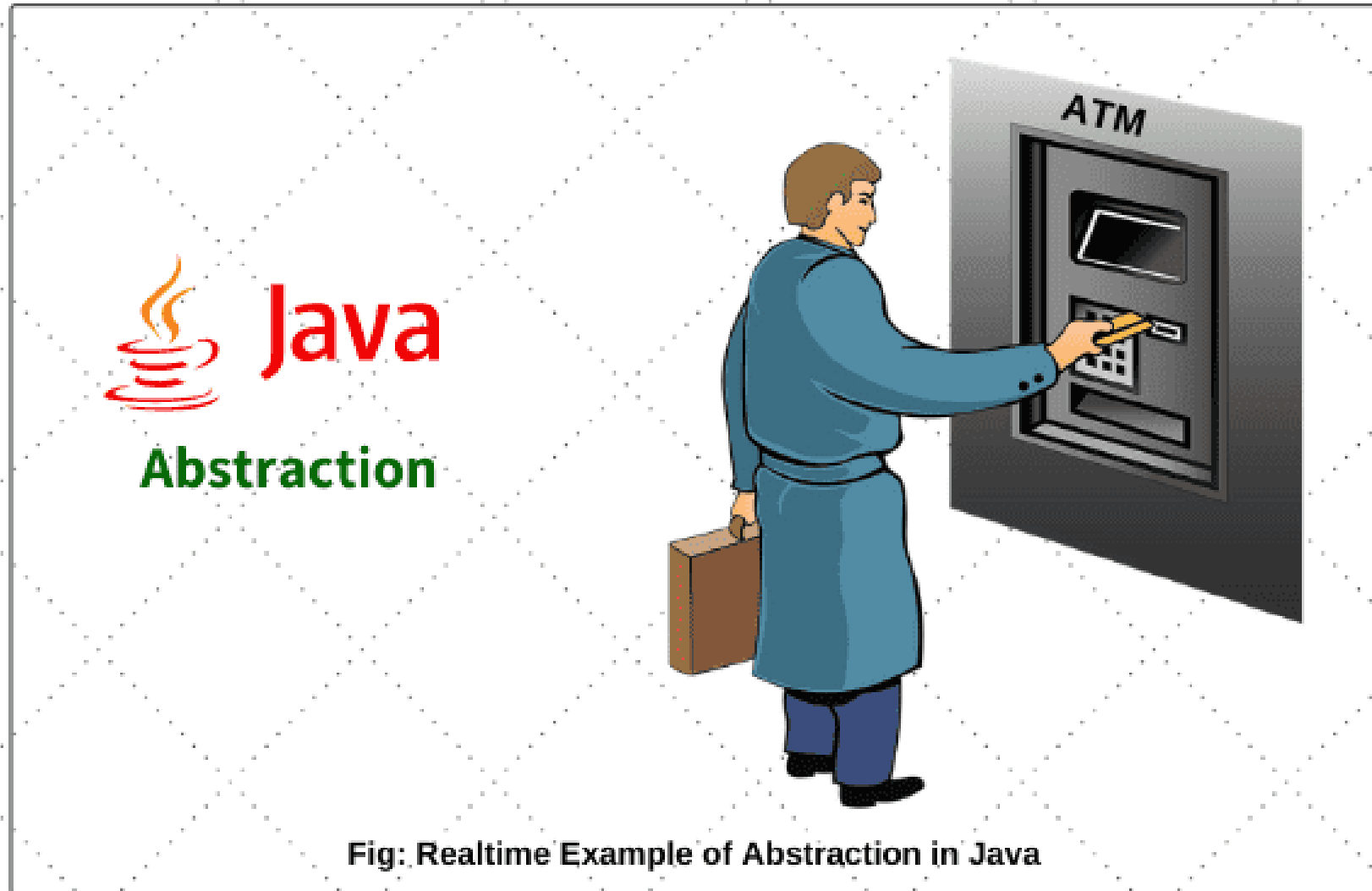
Abstraction

- Another way, it shows only essential things to the user and hides the internal details.
- For example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Examples of

1. ATM machine as a real-time example.
2. A car owner knows how to drive it. He knows about various components of car and how to use them but not how these components are working.
3. When you need to send SMS from your mobile

Abstraction



Abstract Class

1. Abstraction is achieved by abstract classes in OOP.
2. A class which is declared with the abstract keyword is known as an abstract class in Java.
3. It can have abstract and non-abstract methods (method with the body).
4. It needs to be extended and its method implemented.
5. It cannot be instantiated i.e. we can-not create objects of an abstract class.

Abstract Class

- **Points to Remember about abstract class**
 1. An abstract class must be declared with an abstract keyword.
 2. It can have abstract and non-abstract methods.
 3. It cannot be instantiated.
 4. It can have constructors and static methods also.

Abstract Class

Example of abstract class

abstract class A

{

....

....

}

Abstract Class

- **Abstract Method in Java**
- A method which is declared as abstract and **does not have implementation**(or body) is known as an abstract method.
- Example of abstract method
- **abstract void printStatus();**
- `//no method body and abstract keyword`

// Abstract classes in Java

abstract class Bike

```
{  
    abstract void run();  
}
```

class Honda extends Bike

```
{  
    void run()  
    {  
        System.out.println("running safely");  
    }  
}
```

class TestingAbstraction

```
{  
    public static void main(String args[])  
    {  
        Bike obj = new Honda();  
        obj.run();  
    }  
}
```

Output :

run:

running safely

BUILD SUCCESSFUL (total time: 2 seconds)

// Abstract classes in Java

// Abstract classes in Java

```
abstract class Bike
{
    abstract void run();
}
class Honda extends Bike
{
    void run()
    {
        System.out.println("running
safely");
    }
}
```

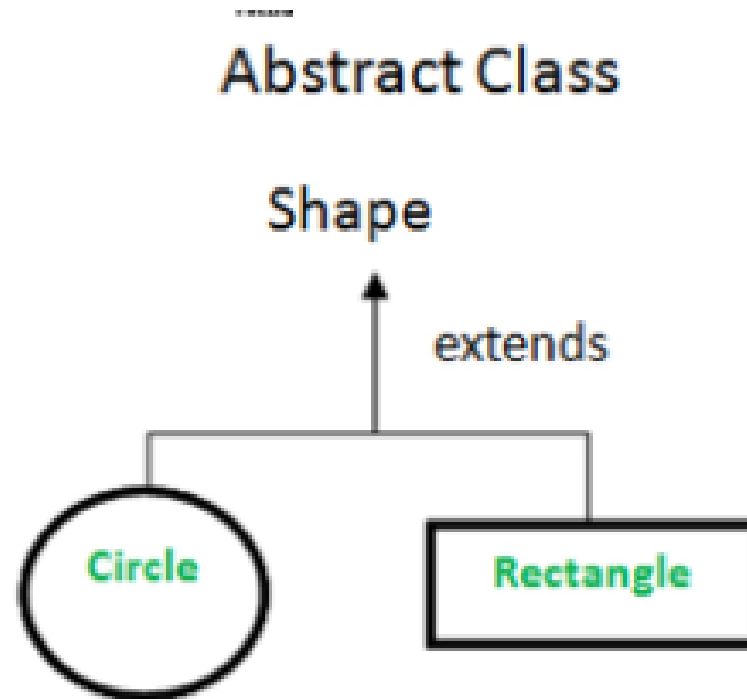
```
class TestingAbstraction
{
    public static void main(String args[])
    {
        Bike obj = new Honda();
        obj.run();
    }
}
```

Output :

```
run:
running safely
BUILD SUCCESSFUL (total time: 2 seconds)
```

Abstract Class

- **Example 2**
- In this example, **Shape** is the abstract class, and its implementation is provided by the **Rectangle** and **Circle** classes.



```
// Implementing abstraction example2
package Abstraction1;
abstract class Shape
{
    abstract void draw();
}
class Rectangle extends Shape
{
    void draw()
    {System.out.println("drawing rectangle");}
}
class Circle1 extends Shape
{
    void draw()
    {System.out.println("drawing circle");}
}
```

```
class TestAbstraction1
{
    public static void main(String args[])
    {
        Shape s=new Circle1();
        s.draw();
        s=new Rectangle();
        s.draw();
    }
}
```

Output

```
run:
drawing circle
drawing rectangle
```


// Implementing abstraction example2

```
// Implementing abstraction example2
package Abstraction1;
abstract class Shape
{
    abstract void draw();
}
class Rectangle extends Shape
{
    void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape
{
    void draw(){System.out.println("drawing circle");}
}
class TestAbstraction1
{
    public static void main(String args[])
    {
        Shape s=new Circle1();
        s.draw();
        s=new Rectangle();
        s.draw();
    }
}
```

Output

```
run:
drawing circle
drawing rectangle
BUILD SUCCESSFUL (total time: 0 seconds)
```

Final Variables and Methods

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context.

1. Final variable
2. Final method
3. Final class

Final Variables

- **Java final variable**
- If you make any variable as final, you cannot change the value of final variable(It will be constant).
- A final variable can be explicitly initialized only once.

Code

```
// Final Variables
package Finals;
class Bike{
    final int speedlimit=90;
    //final variable
    void run()
    {
        speedlimit=400;
    }
}
```

```
}//end of class
public class FinalVariableTesting1 {
    public static void main(String args[])
    {
        Bike obj=new Bike();
        obj.run();
    }
}
```

```
run:
Exception in thread "main" java.lang.RuntimeException:
Uncompilable source code - cannot assign a value to final variable speedlimit
at Finals.Bike.run(FinalVariableTesting1.java:7)
```

Java final method

- **If you make any method as final, you cannot override it.**
- A final method **cannot be overridden** by any subclasses.
- As mentioned previously, the final modifier prevents a method from being modified in a subclass.

```
///Example of final method
package Finals;
class Bike
{
    final void run()
    {
        System.out.println("running");
    }
}
class Honda extends Bike{
    void run()
    {
        System.out.println("running safely with
100kmph");
    }
}
```

```
public class FinalMethodTesting1 {
    public static void main(String args[]){
        Honda honda= new Honda();
        honda.run();
    }
}
```

Exception in thread "main" java.lang.

VerifyError: class Finals.Honda overrides final method run.()

///Example of final method

package Finals;

class Bike

{

 final void run()

 {

 System.out.println("running");

 }

}

class Honda extends Bike{

 void run()

 {

 System.out.println("running safely with 100kmph");

 }

}

public class FinalMethodTesting1 {

 public static void main(String args[]){

 Honda honda= new Honda();

 honda.run();

 }

}|

Exception in thread "main" java.lang.

VerifyError: class Finals.Honda overrides final method run.()

Final Methods

///
//Example of final method

```
package Finals;
```

```
class Bike
```

```
{
```

```
    final void run()
```

```
    {
```

```
        System.out.println("running");
```

```
    }
```

```
}
```

```
class Honda extends Bike{
```

```
    void run()
```

```
    {
```

```
        System.out.println("running safely with 100kmph");
```

```
    }
```

```
}
```

```
public class FinalMethodTesting1 {
```

```
    public static void main(String args[]){
```

```
        Honda honda= new Honda();
```

```
        honda.run();
```

```
    }
```

```
}
```

Exception in thread "main" java.lang.

VerifyError: class Finals.Honda overrides final method
run.()

Java final class

- **If you make any class as final, you cannot extend it.**
- If you will create a subclass of it, compiler will show an error message.
- It means declaration of that class is final.

Example

```
final class Bike
```

```
{
```

```
    .....
```

```
}
```

//Example of final class

```
package Finals;
```

```
final class Bike
```

```
{  
    final void run(){System.out.println("running");}  
}
```

```
class Honda extends Bike
```

```
{
```

```
}
```

```
public class FinalMethodTesting1 {
```

```
    public static void main(String args[]){
```

```
        Honda honda= new Honda();
```

```
        honda.run();
```

```
    }
```

```
}
```

```
|
```

```
Q . . .
```



Error

//Example of final class

```
package Finals;
```

```
final class Bike
```

```
{  
    final void run(){System.out.println("running");}  
}
```

```
class Honda extends Bike
```

```
{
```

```
}
```

```
public class FinalMethodTesting1 {
```

```
    public static void main(String args[]){
```

```
        Honda honda= new Honda();
```

```
        honda.run();
```

```
    }
```

```
}
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

```
|
```

Error

Output

Exception in thread "main" java.lang.VerifyError:

Cannot inherit from final class

Code : //Example of final class

```
//Example of final class
package Finals;
final class Bike
{
    final void
run(){System.out.println("running");}
}
class Honda extends Bike
{

}

public class FinalMethodTesting1 {
```

```
    public static void main(String args[]){
Honda honda= new Honda();
honda.run();
}
}
```

Output

```
Exception in thread "main"
java.lang.VerifyError:
Cannot inherit from final class
```

Summary

1. Final variable – Can not be changed
2. Final method – Can not be overridden
3. Final class – Can not be extended/Inherited